

Estudo Dirigido 1.3

1º Bimestre

Descrição

Encapsulamento é um dos pilares da Programação Orientada a Objetos. Ele consiste em proteger os dados internos de uma classe, controlando como esses dados podem ser acessados ou alterados. Em vez de deixar os atributos livres para qualquer parte do programa modificar, a classe passa a oferecer uma forma segura e organizada de acesso.

Getters são métodos usados para ler o valor de um atributo privado. Eles permitem consultar informações do objeto sem expor diretamente sua estrutura interna.

Setters são métodos usados para alterar o valor de um atributo privado. Com eles, é possível aplicar validações e regras antes de aceitar a mudança, evitando estados inválidos no objeto.

Em Dart, o encapsulamento normalmente é feito com atributos privados, usando `_`, e com métodos tradicionais ou com a sintaxe de `get` e `set`.

```
Exemplo 1

class Heroi {
  String _nome = "";
  int _vida = 100;

  String getNome() {
    return _nome;
  }

  void setNome(String nome) {
    if (nome.isNotEmpty) {
      _nome = nome;
    }
  }

  int getVida() {
    return _vida;
  }

  void setVida(int vida) {
    if (vida >= 0 && vida <= 100) {
      _vida = vida;
    }
  }
}

void main() {
  Heroi h = Heroi();

  h.setNome("Batman");
  h.setVida(95);

  print("Herói: ${h.getNome()}");
  print("Vida: ${h.getVida()}");
}
```

```
Exemplo 1

class Heroi {
  String _nome = "";
  int _vida = 100;

  String get nome => _nome;

  set nome(String valor) {
    if (valor.isNotEmpty) {
      _nome = valor;
    }
  }

  int get vida => _vida;

  set vida(int valor) {
    if (valor >= 0 && valor <= 100) {
      _vida = valor;
    }
  }
}

void main() {
  Heroi h = Heroi();

  h.nome = "Homem-Aranha";
  h.vida = 88;

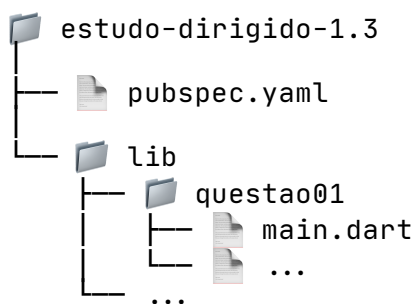
  print("Herói: ${h.nome}");
  print("Vida: ${h.vida}");
}
```

Instruções de entrega

Para a submissão do trabalho, siga os passos abaixo:

- Criar um novo repositório no github chamado **programacao-oo**
 - Selecionar para criar o README.md
 - Selecionar para criar .gitignore usando template Dart
- Incluir o **nome dos integrantes** da equipe no README.md (até 3 alunos)
- Compartilhar o do repositório na **planilha** disponibilizada pelo professor (TEAMS)

Para garantir organização, o trabalho deve seguir a estrutura de diretórios abaixo:



Critérios de avaliação

A avaliação será baseada nos seguintes aspectos:

- Uso correto de classes e atributos
- Implementação de construtores
- Implementação de métodos
- Organização e legibilidade do código
- Capacidade de explicar o funcionamento do código

Questões

Questão 1 – Cliente

Crie uma classe Cliente com os atributos privados `_nome`, `_email`, `_saldoCarteira` e `_ativo`.

Implemente um construtor para inicializar os atributos da classe. Crie getters para todos os atributos e setters com validação. O nome e o email não podem ser vazios. O saldo da carteira não pode ser negativo. O atributo `_ativo` deve ser do tipo `bool`. Implemente os métodos `ativar()`, `desativar()`, `adicionarSaldo(double valor)`, `debitarSaldo(double valor)` e `exibirCliente()`.

No main(), crie dois clientes, altere alguns dados usando setters, adicione saldo em um deles, debite saldo do outro e exiba os dados finais.

Questão 2 – ListaCliente

Crie uma classe ListaCliente para armazenar objetos do tipo Cliente.

Essa classe deve ter uma estrutura interna privada para guardar os clientes. Implemente os métodos inserir(Cliente cliente), remover(int i), tamanho() e get(int i).

O método remover(int i) só deve funcionar se o índice informado for válido. O método get(int i) deve retornar o cliente da posição informada, também apenas se o índice for válido. Crie ainda um método exibirLista() para mostrar todos os clientes armazenados.

No main(), crie alguns objetos Cliente, insira esses objetos em uma ListaCliente, remova um cliente pelo índice, mostre o tamanho da lista, recupere um cliente com get(i) e exiba o conteúdo final da estrutura.

🚩 Importante 🚩

Utilize neste exercício a classe List<> do Dart para armazenar os dados na classe ListaCliente. Exemplo de Código:

```
Exemplo 1

void main() {
  List<String> nomes = [];

  // adicionando
  nomes.add("Ana");
  nomes.add("Bruno");
  nomes.add("Carlos");
  print(nomes);

  // consultando a lista inteira
  for (String nome in nomes) {
    print(nome);
  }

  // pegando um elemento pelo índice
  print(nomes[1]);

  // total de elementos
  print(nomes.length);

  // removendo pelo índice
  nomes.removeAt(0);
  print(nomes);
}
```

Questão 3 – Produto

Crie uma classe Produto com os atributos privados `_nome`, `_preco`, `_estoque` e `_ativo`.

Implemente um construtor para inicializar os atributos da classe. Crie getters para todos os atributos e setters com validação. O nome não pode ser vazio. O preço não pode ser negativo. O estoque não pode ser negativo. O atributo `_ativo` deve ser do tipo `bool`. Implemente os métodos `ativar()`, `desativar()`, `reporEstoque(int quantidade)`, `retirarEstoque(int quantidade)` e `exibirProduto()`.

No main(), crie dois produtos, altere alguns dados usando setters, reponha estoque de um produto, retire estoque de outro e exiba os dados finais.

Questão 4 – ListaProduto

Crie uma classe ListaProduto para armazenar objetos do tipo Produto.

Essa classe deve ter uma estrutura interna privada para guardar os produtos. Implemente os métodos `inserir(Produto produto)`, `remover(int i)`, `tamanho()` e `get(int i)`.

O método `remover(int i)` só deve funcionar se o índice informado for válido. O método `get(int i)` deve retornar o produto da posição informada, também apenas se o índice for válido. Crie ainda um método `exibirLista()` para mostrar todos os produtos armazenados.

No main(), crie alguns objetos Produto, insira esses objetos em uma ListaProduto, remova um produto pelo índice, mostre o tamanho da lista, recupere um produto com `get(i)` e exiba o conteúdo final da estrutura.

Questão 5 – ItemCarrinho

Crie uma classe ItemCarrinho com os atributos privados `_produto` e `_quantidade`.

O atributo `_produto` deve ser do tipo `Produto` e `_quantidade` deve ser maior que zero. Implemente um construtor para inicializar os atributos da classe. Crie getters para todos os atributos e setters com validação. Implemente os métodos `calcularSubtotal()` e `exibirItem()`.

No main(), crie um ou dois produtos, depois crie itens de carrinho usando esses produtos. Altere a quantidade com setter e exiba o subtotal de cada item.

Questão 6 – ListaCarrinho

Crie uma classe ListaCarrinho para armazenar objetos do tipo ItemCarrinho.

Essa classe deve ter uma estrutura interna privada para guardar os itens. Implemente os métodos `inserir(ItemCarrinho item)`, `remover(int i)`, `tamanho()` e `get(int i)`.

O método `remover(int i)` só deve funcionar se o índice informado for válido. O método `get(int i)` deve retornar o item da posição informada, também apenas se o índice for válido. Crie ainda um método `exibirLista()` para mostrar todos os itens armazenados.

No main(), crie alguns objetos ItemCarrinho, insira esses objetos em uma ListaCarrinho, remova um item pelo índice, mostre o tamanho da lista, recupere um item com `get(i)` e exiba o conteúdo final da estrutura.

Questão 7 – Carrinho

Crie uma classe Carrinho com os atributos privados `_cliente`, `_itens` e `_aberto`.

O atributo `_cliente` deve ser do tipo `Cliente`, `_itens` deve ser do tipo `ListaCarrinho` e `_aberto` deve ser do tipo `bool`.

Implemente um construtor para inicializar os atributos da classe. Crie getters para todos os atributos e setters com validação. Implemente os métodos `adicionarItem(ItemCarrinho item)`, `removerItem(int i)`, `calcularTotal()` e `exibirCarrinho()`.

No main(), crie um cliente, crie alguns produtos, monte itens de carrinho e adicione esses itens em um objeto Carrinho. Depois, exiba todos os dados do carrinho e o valor total da compra.

Questão 8 – CupomDesconto

Crie uma classe CupomDesconto com os atributos privados `_codigo`, `_percentual` e `_ativo`.

O atributo `_codigo` não pode ser vazio. O atributo `_percentual` deve estar entre 0 e 100. O atributo `_ativo` deve ser do tipo `bool`. Implemente um construtor para inicializar os atributos da classe. Crie getters para todos os atributos e setters com validação. Implemente os métodos `ativar()`, `desativar()`, `calcularDesconto(double valor)` e `exibirCupom()`.

No main(), crie um cupom, altere alguns dados com setter, teste o cálculo do desconto sobre um valor qualquer e exiba os dados finais.

Questão 9 – Pedido

Crie uma classe Pedido com os atributos privados `_numero`, `_carrinho`, `_cupom`, `_totalBruto`, `_totalFinal` e `_status`.

O atributo `_numero` não pode ser vazio. O atributo `_carrinho` deve ser do tipo `Carrinho`. O atributo `_cupom` pode ser nulo ou do tipo `CupomDesconto`. Os atributos `_totalBruto` e `_totalFinal` não podem ser negativos. O atributo `_status` deve guardar valores como "aberto", "fechado" ou "cancelado".

Implemente um construtor para inicializar os atributos da classe. Crie getters para todos os atributos e setters com validação. Implemente os métodos `fecharPedido()`, `cancelarPedido()` e `exibirResumoPedido()`.

Ao fechar o pedido, calcule o total bruto com base no carrinho. Se houver cupom ativo, calcule também o total final com desconto. Caso não haja cupom, o total final deve ser igual ao total bruto.

No main(), crie um carrinho com itens, associe um cupom ao pedido, feche o pedido e exiba o resumo final.

Questão 10 – Loja

Crie uma classe Loja com os atributos privados `_nome`, `_clientes`, `_produtos` e `_pedidos`.

O atributo `_nome` não pode ser vazio. O atributo `_clientes` deve ser do tipo `ListaCliente`. O atributo `_produtos` deve ser do tipo `ListaProduto`. O atributo `_pedidos` pode ser uma estrutura de armazenamento que você criar para guardar objetos do tipo `Pedido`, ou uma `List<Pedido>` se isso for permitido.

Implemente um construtor para inicializar os atributos da classe. Crie getters para todos os atributos e setters com validação. Implemente os métodos `cadastrarCliente(Cliente cliente)`, `cadastrarProduto(Produto produto)`, `registrarPedido(Pedido pedido)` e `exibirResumoLoja()`.

No `main()`, monte um cenário simples com clientes, produtos e pedidos. Cadastre os clientes e produtos na loja, registre pelo menos um pedido e exiba um resumo geral.